

Build Agentic Workflows That Work

Professor Peyman Teymoori



Contents

”

01 | Kickoff & Recap

02 | Pick Your Pattern

03 | Design the Blueprint

04 | Build in n8n

05 | Test Like a Pro

06 | Harden & Polish

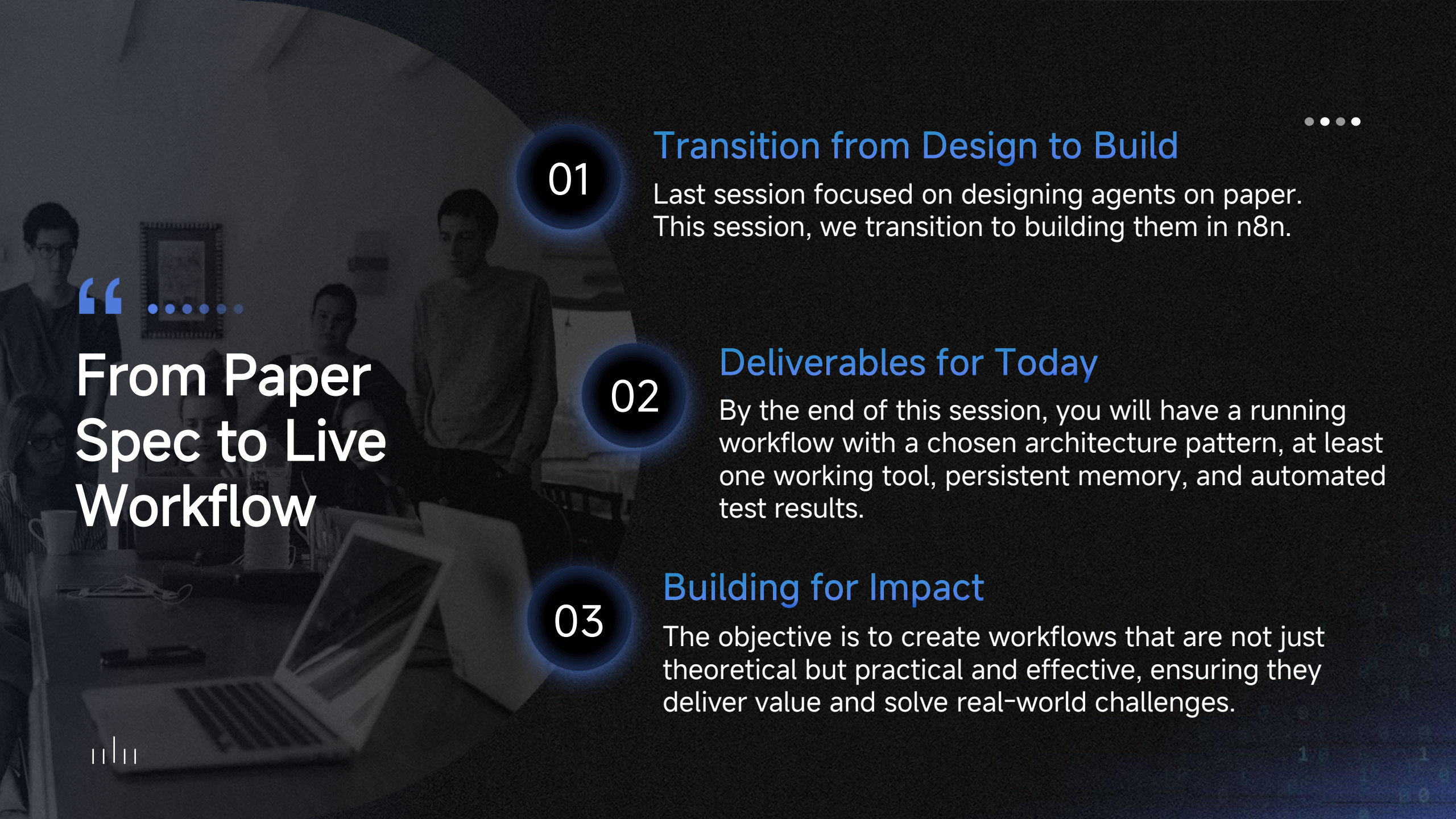
07 | Next Frontiers



01

Kickoff & Recap





“.....

From Paper Spec to Live Workflow

01

Transition from Design to Build

Last session focused on designing agents on paper. This session, we transition to building them in n8n.

02

Deliverables for Today

By the end of this session, you will have a running workflow with a chosen architecture pattern, at least one working tool, persistent memory, and automated test results.

03

Building for Impact

The objective is to create workflows that are not just theoretical but practical and effective, ensuring they deliver value and solve real-world challenges.





Role Rotation Refresher



▶ Role Rotation Overview

Teams rotate roles to broaden skills. Last session's Product Owner becomes Agent Architect, Architect becomes Toolsmith, Toolsmith becomes Evaluator, and Evaluator becomes Product Owner.

▶ Role Responsibilities

New PO ensures tech choices align with business goals, AA designs the workflow, TS builds tools, EV automates tests and tracks metrics. This rotation ensures a well-rounded team effort.



02

Pick Your Pattern



“.....

Why Architecture Patterns Matter

01

Patterns and Task Structure

Patterns match task structure to agent logic. Simple classification needs direct response, multi-tool reasoning needs ReAct, multi-phase projects need plan-and-execute, and specialist domains need multi-agent teams.

||||

02

Choosing the Right Pattern

- Prevents over-engineering and keeps workflows debuggable.
- Ensures that the workflow is as simple as possible while still meeting all requirements.





Direct Response Pattern

Use Case

01

Use when the task is single-step, deterministic, and fast. Input flows through fixed logic to output without loops.

Ideal Scenarios

02

Ideal for email classification, keyword extraction, or policy lookups with known rules. It is straightforward and efficient for these types of tasks.

Pros and Cons

03

Pros: speed and predictability; cons: no adaptation or tool choice. It is best suited for tasks that do not require flexibility or multiple tools.

“

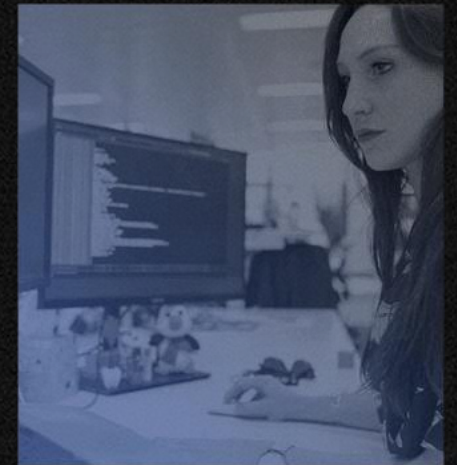
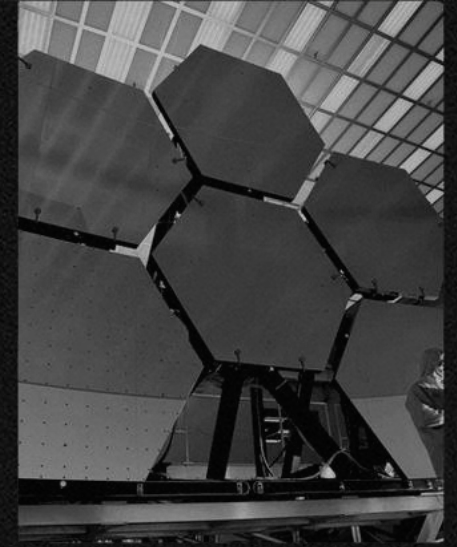
ReAct Loop Explained

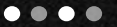
- ReAct Cycle

ReAct cycles through thought, action, and observation until the goal is met. The agent decides which tool to call, executes it, observes the result, and re-evaluates.

- Best Use Cases

Perfect for Q&A with lookup, adaptive research, or any task where the next step depends on previous findings. It allows for dynamic decision-making.





Plan-and-Execute Workflow



► Workflow Structure

Planner breaks the request into ordered steps, executor runs each step, and synthesizer merges outputs into a final answer.

► Ideal Applications

Best for complex reports, data pipelines, or any task with distinct phases that benefit from upfront visibility and auditability.



► Benefits

Provides a clear structure and progress tracking, making it easier to optimize and debug complex workflows.

“

...

Pattern Decision Tree

► Decision Process

- Start by asking if the task is single-step deterministic.
- If yes, choose direct response; if no, check whether multiple tools, multi-phase structure, or specialist sub-agents are needed.

03

Design the Blueprint



“

Workflow Diagram First

...



► Sketching the Workflow

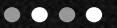
Sketch the entire n8n canvas before touching nodes: trigger, preprocessing, agent core, tool routes, memory saves, and output formatting.



► Template Structure

Use the template: trigger → validate → load memory → AI think → switch → tools → save → loop → respond.
A clear diagram prevents wiring mistakes.





Tool Contract Template

01

Tool Requirements

Every tool needs a name, purpose, input schema, output schema, and error cases. Define required versus optional parameters.

02

Example and Validation

Provide examples and specify timeout and retry rules. Clear contracts prevent integration bugs and make mock testing easy during development.

03

Importance of Contracts

Tool contracts ensure that each component works seamlessly with others, reducing the risk of integration issues.

“

Memory Design Choices

01

Memory Options

Workflow variables suffice for short sessions;
Postgres or Redis stores persistent history.

02

Retention Policies

Decide what to store—conversation turns, tool outputs, confidence scores—and what never to store such as PII. Set retention policies to comply with privacy rules and keep storage costs low.



“

....

Error Handling Layers

01

Input Validation

Validate inputs at the start to prevent downstream nodes from wasting API calls or storage.

Node Failure Management

Catch node failures with error triggers and set max iterations on loops.
Return safe fallback messages instead of raw errors to maintain user trust.

02

03

Logging and User Experience

Log technical details internally while showing friendly summaries externally.
This ensures that issues are tracked without overwhelming users.



04

Build in n8n





Webhook Trigger Setup



01

Creating the Webhook

Create a POST webhook at a unique path; expect JSON with query and session_id.

02

Immediate Validation

Add immediate input validation: check query length, data types, and generate a session_id if missing. Early validation prevents downstream issues.



“ ”

Implement Your First Tool

...

01

Tool Implementation

Start with an HTTP request to a real API or a Code node that returns mock data matching your contract.

02

Node Configuration

Configure method, URL, authentication, timeout, and retry count. Test the node in isolation before wiring it into the switch router.

03

Schema Matching

Ensure outputs match the promised schema to avoid integration issues and ensure smooth workflow execution.





Wire the ReAct Loop



01

▶ Connecting Nodes

Connect AI think node to switch node, route to chosen tool, save observation to memory, then loop back until the ANSWER tool is selected.

02

▶ Loop Control

Set max iterations to five to prevent infinite loops and add a break condition that checks for the ANSWER tool name in the JSON output.





Save and Load Memory



01

Memory Nodes

Use Postgres nodes to insert each step into conversation_steps and to load the last ten turns before every think node.

02

Session Metadata

Store session metadata such as start time and total tokens so you can track cost and latency across multiple turns without re-computing statistics.

03

Memory Importance

Persistent memory ensures that the workflow maintains context across steps, improving user experience and reducing redundant processing.





Format Final Response



Response Structure

Extract the answer text, append citations, add confidence score, and wrap in a user-friendly JSON payload.



Session Tracking

Include session_id for traceability and a flag indicating whether human review is recommended. Keep the format consistent for reliable parsing.



05

Test Like a Pro



“

....

Automate Five Must-Pass Tests

Evaluation Workflow

Create a separate evaluation workflow that loops over your original test cases, calls the main agent webhook, and checks expected content, citations, latency, and cost.

Continuous Improvement

Automated testing ensures that any changes do not break existing functionality and helps in maintaining high quality standards.

Recording Results

Record pass-fail status and reasons for failure so you can regression-test after every change without manual effort.



06

Harden & Polish



“.....

Add Timeout Limits

...

Timeouts and Loops

Set thirty-second timeouts on HTTP requests and five-step caps on loops to prevent runaway costs.

Graceful Messages

Return a graceful message when limits are hit, log the truncated session, and flag for human review so users understand what happened.

User Trust

Maintaining user trust through clear communication about limits and next steps is crucial for a positive user experience.





Secure Credentials



► Credential Management

Store API keys in n8n's credential manager, never hard-code them in nodes. Use header or bearer authentication.

► Access Control

Restrict webhook endpoints to trusted IP ranges or require header tokens to prevent unauthorized usage that could inflate bills.



07

Next Frontiers





Lecture 3 Preview

Upcoming Features

Next session adds orchestration with state machines, full guardrails including schema validation and allow-lists, structured logging for traces, and recovery strategies.

Preparation

Bring two working tools and a list of current failure modes. Be ready to dive into advanced features that enhance reliability and safety.

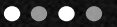
Continuous Learning

Continuous learning and iteration are key to building robust and efficient workflows that meet evolving business needs.





Key Takeaways Today



Tool Implementation

Implement one solid tool before adding more. This ensures that each component works reliably and contributes to the overall workflow.

Continuous Testing

Log everything and test adversarially. Continuous evaluation turns gut feelings into measurable improvements that stakeholders trust.

01

Pattern Selection

Choose the simplest pattern that fits your needs. Over-engineering can complicate workflows and make them harder to debug.

03

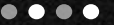
Early Validation

Validate inputs early to prevent downstream issues. This saves time and resources by catching errors at the earliest possible stage.

04



Further Resources



▶ Resource Access

Access n8n workflow templates, mock tool code snippets, and evaluation workflows in the course drive.

Recommended reading includes the n8n advanced tutorial and OpenAI function-calling best-practice docs.

▶ Online

<https://github.com/peyman-t/AgenticAI>



THANK YOU

